

Highlight Vault Project Documentation

1. Project Summary

Highlight Vault, oyuncuların video kliplerini yükleyip saklayabileceği, paylaşabileceği ve diğer kullanıcıların feed/explore üzerinden keşfedebileceği bir gaming clip platformudur.

Platformun ana hedefi sadece genel video paylaşımı yapmak değil, oyun odaklı bir sosyal klip deneyimi sunmaktır. Bu yüzden upload, post, feed, explore, library ve moderation akışlarının merkezinde "bu içerik oyunla alakalı mı, public olarak gösterilmeli mi, yoksa insan moderator kontrolüne mi düşmeli?" sorusu vardır.

Ana roller:

- ? Normal kullanıcı: Klip yükler, library'de saklar, post olarak paylaşır, feed/explore kullanır.
- ? Admin/moderator: Kullanıcıları yönetir, moderation queue'daki klipleri inceler, approve/reject/remove kararlarını verir.
- ? Otomatik moderation sistemi: FFmpeg ile videodan kare çıkarır, OpenAI ile görsel/semantik kontrol yapar, riskli içerikleri public yayınlamadan önce durdurur.

2. Main Features

- ? User register/login
- ? Profile and profile photo support
- ? Clip upload and personal library
- ? Cloudinary video storage integration
- ? Feed and explore pages
- ? Post creation from clips
- ? Like, favorite, comment and playlist flows
- ? Admin user management
- ? AI-assisted clip moderation
- ? Manual moderation queue
- ? Safe user-facing error messages through toaster UI

3. Technology Stack

Frontend:

- ? Angular
- ? TypeScript
- ? Angular routing
- ? Angular services for API access
- ? Shared components such as sidebar, toast, dialogs and dropdowns

Backend:

- ? Spring Boot
- ? Java
- ? Spring MVC controllers
- ? Service layer
- ? Repository layer
- ? JPA entities plus native @Query methods where complex SQL is needed

Database:

- ? MySQL style relational schema
- ? Tables for users, clips, posts, comments, favorites, reports, moderation results and playlists

Media and AI:

- ? Cloudinary for video/image hosting
- ? FFmpeg for extracting frames from videos
- ? OpenAI Responses API for visual/semantic moderation decisions

4. High Level Architecture

```

text
Angular Frontend
|
| REST API
v
Spring Boot Backend
|
| JPA repositories / native queries
v
MySQL Database

```

Backend side integrations:

- Cloudinary video URLs are stored on clips
- FFmpeg extracts frames from video URLs
- OpenAI receives extracted frames plus clip metadata
- Moderation results are stored for audit/history

5. Frontend Structure

Important frontend folders:

- ? frontend/src/app/features: Page-level feature components.
- ? frontend/src/app/core/services: API service classes.
- ? frontend/src/app/core/models: Shared frontend models.
- ? frontend/src/app/shared: Reusable UI components such as sidebar, toast, dropdowns and dialogs.
- ? frontend/src/app/app.routes.ts: Main Angular route definitions.

Important pages/features:

- ? feed: Public/social feed of shared posts.
- ? explore: Discoverable public posts.
- ? library: User's own uploaded clips.
- ? clip-editor: Upload/edit flow for clips.
- ? add-post: Share a clip as a post.
- ? profile: User profile and profile content.
- ? users-list-page: Admin user management.
- ? moderation: Admin moderation queue for flagged clips.

Frontend API services:

- ? clip.service.ts: Clip CRUD, upload response handling and scan trigger.
- ? moderation.service.ts: Moderation queue and moderator decision endpoints.
- ? explore.service.ts: Explore/feed related post operations.
- ? auth.service.ts: Current user/admin state and auth data from local storage.

6. Backend Structure

Important backend folders:

- ? controller: REST endpoints.
- ? service: Business logic and moderation workflows.

- ? repository: Database access with JPA/native queries.
- ? entity: JPA entity classes.
- ? dto: Request/response DTOs.
- ? enums: State machines such as moderation status and visibility status.

Typical request flow:

```
text
Angular component
-> Angular service
-> Spring Controller
-> Spring Service
-> Repository
-> Database
```

Example:

```
text
Moderation page
-> ModerationService.getQueue()
-> GET /api/moderation/queue
-> ModerationController
-> ModerationService
-> ClipRepository.findModerationQueue()
-> clips table
```

7. Main Database Tables

The exact schema can evolve, but the project currently revolves around these tables.

users

Stores registered users.

Important fields:

- ? id: Primary key.
- ? username: Unique username.
- ? email: User email.
- ? password_hash: Hashed password.
- ? profile_photo_url: User avatar/profile photo.
- ? isAdmin / isadmin: Admin role flag depending on query naming.

Purpose:

- ? Identifies clip owners, post authors, moderators and normal users.

clips

Stores uploaded video clips.

Important fields:

- ? id: Primary key.
- ? user_id: Owner/uploader.
- ? title: Clip title.
- ? notes: Optional clip notes.
- ? video_url: Cloudinary/video URL.

- ? thumbnail_url: Thumbnail URL.
- ? created_at: Creation time.
- ? is_deleted: Soft-delete/trash flag.
- ? visibility_status: Whether the clip can be shown publicly.
- ? moderation_status: Moderation lifecycle state.
- ? moderation_score: Final risk/review score stored on the clip.
- ? moderation_reason: Human-readable reason from scanner/admin.
- ? moderation_checked_at: Last moderation scan time.
- ? reviewed_by: Moderator user id.
- ? reviewed_at: Moderator decision time.
- ? removed_reason: Reason for removal.
- ? removed_at: Removal time.

Purpose:

- ? Source of truth for uploaded videos.
- ? Feed/explore queries only include clips that are public and approved/auto-approved.

posts

Stores shared posts created from clips.

Important fields:

- ? id: Primary key.
- ? clip_id: Shared clip.
- ? user_id: Post author.
- ? caption / content: Post text/caption depending on implementation.
- ? created_at: Creation time.

Purpose:

- ? A clip can exist privately in library, but a post is what appears in social areas.

comments

Stores comments under posts.

Important fields:

- ? id
- ? post_id
- ? user_id
- ? content
- ? parent_comment_id
- ? created_at

Purpose:

- ? Supports normal comments and replies.

favorites / likes

Stores user interactions.

Purpose:

- ? Likes affect post engagement.
- ? Favorites allow users to save clips/posts.

games and tags

Stores game/category metadata and user-facing tags.

Purpose:

- ? Filtering clips by game/tag.
- ? Giving AI moderation additional context.

clip_tags

Join table between clips and tags.

Purpose:

- ? A clip can have multiple tags.
- ? Tags are also included in moderation context.

reports

Stores user reports for inappropriate content.

Important fields usually include:

- ? target type
- ? target id
- ? reason
- ? status
- ? reporter
- ? created time

Purpose:

- ? Manual/community moderation input.

moderation_results

Stores audit results from scanners/providers.

Important fields:

- ? target_type: Usually CLIP.
- ? target_id: Clip id.
- ? provider: Example METADATA_PRECHECK, OPENAI_CLIP, OPENAI_THUMBNAIL, FFMPEG.
- ? category: Main moderation category.
- ? score: Provider-specific score.
- ? flagged: Whether the provider thought the content should be reviewed.
- ? raw_result: Structured JSON audit payload.
- ? created_at: Audit creation time.

Purpose:

- ? Keeps a history of why a clip was approved, flagged or reviewed.
- ? Useful for debugging AI moderation behavior without exposing raw provider errors to users.

8. Important Enums and State Fields

VisibilityStatus

Current enum values:

- ? PRIVATE: Clip belongs to user and is not public.
- ? PUBLIC: Clip can be shown publicly.
- ? LIMITED: Restricted visibility state reserved for future use.

? HIDDEN: Hidden from public areas, usually because moderation requires review or rejected it.

? REMOVED: Removed from normal platform visibility.

ModerationStatus

Current enum values:

? DRAFT: Clip is not ready for public review/publishing.

? PENDING_REVIEW: Waiting for review.

? AUTO_APPROVED: Automatically approved by scanner.

? NEEDS_MANUAL_REVIEW: Needs admin/moderator decision.

? APPROVED: Approved by moderator.

? REJECTED: Rejected by moderator.

? REMOVED: Removed by moderator/admin.

? APPEALED: User appealed a moderation decision.

ModerationActionType

Current enum values:

? APPROVE

? REJECT

? REMOVE

? RESTORE

? HIDE

? LIMIT

? RESOLVE_REPORT

? DISMISS_REPORT

The current moderation queue UI primarily uses:

? APPROVE: Makes clip public and approved.

? REJECT: Marks clip rejected and hidden.

? REMOVE: Marks clip removed and records removal reason/time.

9. Clip Upload Flow

Current intended flow:

text

User selects video in frontend

-> video is uploaded/stored

-> frontend sends clip metadata/video URL to backend

-> backend creates clips row

-> frontend calls /api/clips/scan/{clipId}

-> backend runs moderation scan

-> frontend shows toaster result

-> user is sent back to library

Important behavior:

? Upload and scan are separated so a failed/slow scan does not break clip creation.

? If scan cannot finish, the user receives a toaster message and the clip can be kept private/manual-review instead of crashing the upload flow.

? Public feed/explore should only show clips that are both public and approved/auto-approved.

10. Post / Feed / Explore Flow

Library is where uploaded clips live.

Feed/explore show public posts.

For a clip to appear in public social areas, these conditions matter:

- ? The clip must not be deleted.
- ? visibility_status should be PUBLIC.
- ? moderation_status should be APPROVED or AUTO_APPROVED.
- ? A post should exist for the clip if the page is post-based.

This prevents private, hidden, rejected, removed or review-waiting clips from leaking into public surfaces.

11. Moderation System Overview

The moderation pipeline is implemented mainly in:

- ? ModerationScannerService
- ? OpenAiVisualModerationService
- ? FfmpegFrameExtractionService
- ? ModerationService
- ? ModerationController
- ? ClipRepository
- ? ModerationResultRepository

Main scan flow:

```
text
scanClipForUpload(clipId)
-> clean visibility target: PRIVATE

scanClipForPublishing(clipId, caption)
-> clean visibility target: PUBLIC

scanClip(...)
-> load clip
-> run metadata precheck
-> save METADATA_PRECHECK audit result
-> extract video frames with FFmpeg
-> if frames exist, send frames + metadata to OpenAI
-> otherwise fallback to thumbnail AI scan
-> combine metadata result and AI result
-> adjust visibility depending on upload/publish context
-> save final moderation fields on clip
```

12. FFmpeg Frame Extraction

The backend does not send the full video file to OpenAI. Instead:

- ? FFmpeg extracts a small number of frames from the video.
- ? The frames are converted to image/data URLs.
- ? OpenAI receives those images plus metadata.

Why this approach is useful:

- ? Cheaper than sending full video.
- ? Faster than full video analysis.
- ? Good enough to catch many obvious off-topic or unsafe uploads.

Limitation:

- ? If the unsafe content appears only between extracted frames, the scan may miss it.
- ? More frames improve coverage but increase latency and AI cost.

13. OpenAI Moderation Output

The AI service asks OpenAI for strict JSON with this schema:

```
json
{
  "decision": "APPROVE_PUBLIC | NEEDS_REVIEW | REJECT_PUBLIC",
  "confidence": 0,
  "risk_score": 0,
  "game_relevance": "CLEAR_GAMEPLAY | GAME_RELATED_CREATOR_CONTENT | AMBIGUOUS | NOT_GAMING",
  "safety_severity": "SAFE | LOW | MEDIUM | HIGH | SEVERE",
  "primary_category": "CLEAN_GAMING | AMBIGUOUS_GAME_RELEVANCE | OFF_TOPIC_NON_GAMING | REAL_WORLD_POLITICAL_PROPAGANDA | EXTREMIST_OR_HATE | REAL_WORLD_VIOLENCE | SEXUAL_CONTENT | SELF_HARM | ILLEGAL_ACTIVITY | HARASSMENT | OTHER_POLICY_RISK",
  "reason": "Human-readable explanation"
}
```

Meaning of fields:

- ? decision: Main AI decision.
- ? confidence: How confident the model is in its decision.
- ? risk_score: AI-estimated risk from 0 to 100.
- ? game_relevance: Whether the clip appears to be actual gaming content.
- ? safety_severity: Safety severity of the detected issue.
- ? primary_category: Most important category for the decision.
- ? reason: Explanation that can be shown to moderators.

The backend parses this response and stores:

- ? A readable moderation_reason on the clip.
- ? A compact structured JSON audit record in moderation_results.raw_result.

14. How Risk Score Is Calculated

There are two related but different scores:

- ? Provider/audit score in moderation_results.score
- ? Final clip score in clips.moderation_score

This distinction is important.

14.1 Metadata Precheck Score

ModerationScannerService.runMetadataPrecheck(...) currently returns:

```
text
status: AUTO_APPROVED
visibility: PUBLIC
score: 1
flagged: false
category: METADATA_READY
```

Meaning:

- ? Metadata precheck does not currently perform heavy rule checks.
- ? It mainly prepares the flow and creates a low base score.
- ? The base score is 1.

14.2 AI Risk Score

OpenAiVisualModerationService.parseSignal(...) reads:

```
text
risk_score
```

from the OpenAI JSON response.

Then it clamps the value into the 0-100 range:

```
text
score = max(0, min(100, risk_score))
```

Examples:

- ? AI returns risk_score = 89, backend keeps 89.
- ? AI returns risk_score = 120, backend stores 100.
- ? AI returns risk_score = -5, backend stores 0.

The AI audit result is saved into moderation_results.score.

14.3 AI Flagged Logic

The backend marks the AI signal as flagged like this:

```
text
if decision is blank:
    use explicit flagged field if present
else:
    flagged = decision != APPROVE_PUBLIC
```

So:

- ? APPROVE_PUBLIC means not flagged.
- ? NEEDS_REVIEW means flagged.
- ? REJECT_PUBLIC means flagged.

14.4 Combining Metadata and AI Result

The current combine logic:

```
text
if AI category starts with "AI_":
    require manual review with PRIVATE visibility
else if AI is flagged:
    require manual review with HIDDEN visibility
    final score = max(metadata score, AI risk score, 40)
else if metadata/rule result is flagged:
    keep rule result
else:
    AUTO_APPROVED + PUBLIC
    final score = max(metadata score, AI score)
```

Interpretation:

- ? AI_DISABLED, AI_UNAVAILABLE, AI_NOT_CONFIGURED are treated as infrastructure/configuration states because they start with AI_.
- ? Real unsafe categories such as EXTREMIST_OR_HATE or OFF_TOPIC_NON_GAMING are treated as flagged AI results.
- ? Clean AI result becomes AUTO_APPROVED.

14.5 Manual Review Score Floor

When a clip needs manual review because of a real AI safety/content flag, the final result now preserves the provider risk score:

```
text
final score = max(baseResult.score, visualSignal.score, 40)
```

Because the metadata base score is currently 1, a clip with an AI risk score of 89 becomes:

```
text
moderation_results.score = 89
clips.moderation_score = 89
```

This means the moderation queue can show the real AI risk level instead of showing every flagged clip as 40.

For infrastructure/configuration problems such as AI_DISABLED, AI_UNAVAILABLE or AI_NOT_CONFIGURED, there is no meaningful unsafe-content risk score from the provider. Those cases still use the manual-review floor:

```
text
clips.moderation_score = 40
```

Current meaning:

- ? moderation_results.score: What the scanner/provider reported.
- ? clips.moderation_score: The final platform workflow score after combining results.

14.6 Clean Upload Visibility Adjustment

The same scanner is used for upload and publishing, but with different clean visibility targets.

Upload scan:

```
text
scanClipForUpload(clipId)
clean visibility target = PRIVATE
```

If AI passes during upload:

```
text
status remains AUTO_APPROVED
visibility becomes PRIVATE
reason = "AI moderation passed; clip remains private until the user publishes it."
```

Publishing scan:

```
text
scanClipForPublishing(clipId, caption)
clean visibility target = PUBLIC
```

If AI passes during publishing:

```
text
status = AUTO_APPROVED
visibility = PUBLIC
```

This distinction lets users upload a safe clip into their private library without automatically publishing it.

15. Visibility and Moderation Decision Matrix

Current automated scan behavior:

Situation	moderation_status	visibility_status	Reason
Clean upload scan	AUTO_APPROVED	PRIVATE	Safe, but still private until user publishes
Clean publish scan	AUTO_APPROVED	PUBLIC	Safe and publishable
AI unavailable/disabled/not configured	NEEDS_MANUAL_REVIEW	PRIVATE	Scanner could not make reliable decision
AI flagged unsafe/off-topic	NEEDS_MANUAL_REVIEW	HIDDEN	Needs moderator review before public display
FFmpeg unavailable and thumbnail AI unavailable	NEEDS_MANUAL_REVIEW	PRIVATE	No reliable visual decision

Current manual moderator behavior:

Moderator action	moderation_status	visibility_status	Extra fields
APPROVE	APPROVED	PUBLIC	reviewed_by, reviewed_at, clears removal fields
REJECT	REJECTED	HIDDEN	moderation_reason, reviewed_by, reviewed_at
REMOVE	REMOVED	REMOVED	removed_reason, removed_at, reviewed_by, reviewed_at
RESTORE	APPROVED	PUBLIC	Similar to approve

16. Moderation Queue

The moderation queue shows clips whose moderation_status is one of:

- ? PENDING_REVIEW
- ? NEEDS_MANUAL_REVIEW
- ? APPEALED

The queue UI displays:

- ? Thumbnail
- ? Video preview with custom player HUD
- ? Uploader
- ? Status
- ? Score
- ? Category
- ? Created date
- ? AI/moderation reason
- ? Moderator note input
- ? Approve / Reject / Remove buttons

Moderators can filter the queue by:

- ? Status
- ? Minimum risk score
- ? Category
- ? Created date range

Approve vs Reject vs Remove:

- ? Approve: The clip is allowed publicly.
- ? Reject: The upload is denied and hidden, but it remains a moderation decision record.
- ? Remove: The content is treated as removed from the platform and gets removal metadata.

Rejected or hidden clips can still appear in the uploader's library with a locked moderation overlay. If a clip is rejected, the uploader can submit an appeal. An appealed clip is moved back to APPEALED and HIDDEN, then appears in the moderation queue for another review.

17. Error Handling

Important rule:

- ? The frontend should not show raw SQL, stack traces, Cloudinary HTML errors or OpenAI provider errors to users.

Instead:

- ? Backend returns safe response messages.
- ? Frontend maps errors through safe error helpers.
- ? Toast UI shows user-friendly messages.
- ? Technical details should remain in logs or audit tables.

Examples of safe messages:

- ? Username is already taken.
- ? Upload failed because the file is too large.
- ? Clip saved, but it needs moderation review before it can be shared.
- ? Could not refresh moderation queue.

Bad messages to avoid:

- ? SQL statements
- ? Duplicate key SQL text
- ? Raw HTML error bodies from Cloudinary
- ? Full OpenAI response payloads in user-facing UI

18. Important API Endpoints

Clip endpoints:

```
text
POST /api/clips
POST /api/clips/scan/{id}
POST /api/clips/{id}/appeal
GET /api/clips
GET /api/clips?uploaderId={id}
```

Post endpoints:

```
text
POST /api/posts
GET /api/posts/feed
GET /api/posts/explore
```

Moderation endpoints:

```
text
GET
/api/moderation/queue?status={status}&minScore={score}&category={category}&fromDate={yyyy-mm-dd}&toDate={yyyy-mm-dd}
```

```
POST /api/moderation/clips/{id}/decision
POST /api/moderation/clips/{id}/approve
POST /api/moderation/clips/{id}/reject
POST /api/moderation/clips/{id}/remove
POST /api/moderation/clips/{id}/restore
```

User/auth endpoints depend on current controller implementation, but generally include:

```
text
POST /api/auth/login
POST /api/auth/register
GET /api/users/{id}
```

19. Environment and Configuration

Important configuration values:

```
properties
app.moderation.ai.enabled=true
app.moderation.ai.model=gpt-5.4
openai.api-key=${OPENAI_API_KEY}
```

Security recommendation:

- ? Do not commit a real OpenAI API key to application.properties.
- ? Use environment variables locally and in production.
- ? Keep .env or machine-level environment config out of git.

For local development:

```
powershell
$env:OPENAI_API_KEY="your_key_here"
```

Then run backend from the same terminal.

20. Running Locally

Backend:

```
bash
cd backend/app
mvn spring-boot:run
```

Frontend:

```
bash
cd frontend
npm install
npm start
```

Common local URLs:

```
text
Frontend: http://localhost:4200
Backend: http://localhost:8080
```

21. Recommended Test Scenarios

Basic user flow:

- ? Register a user.
- ? Log in.
- ? Upload a normal gaming clip.
- ? Confirm it appears in library.
- ? Share it as a post.
- ? Confirm it appears in feed/explore after approved/public.

Moderation flow:

- ? Upload a clearly gaming clip.
- ? Confirm upload scan does not block library.
- ? Upload an off-topic or unsafe clip.
- ? Confirm it becomes NEEDS_MANUAL_REVIEW.
- ? Open /moderation as admin.
- ? Confirm the clip appears in the queue.
- ? Approve, reject and remove test clips.
- ? Confirm feed/explore only show approved public clips.

Error handling flow:

- ? Try duplicate username/email.
- ? Try too-large upload.
- ? Stop backend and refresh moderation queue.
- ? Confirm user sees safe toaster messages, not raw SQL/provider errors.

22. Known Limitations and Improvement Ideas

Moderation improvements:

- ? Add background job queue for scans instead of doing them during request flow.
- ? Store scan status such as SCANNING, SCAN_FAILED, SCAN_COMPLETE.
- ? Extract more frames for longer videos.
- ? Sample frames across intro/middle/end more deliberately.

Security improvements:

- ? Replace localStorage-only auth assumptions with proper JWT/session validation.
- ? Enforce backend authorization for ownership/admin-only operations.
- ? Add rate limiting for uploads and moderation actions.
- ? Validate video MIME type and size on backend.

Frontend improvements:

- ? Create a reusable video player component instead of copying HUD logic.
- ? Create shared moderation status badges.
- ? Improve empty/error states on admin pages.

Backend/database improvements:

- ? Add stronger DB migrations using Flyway or Liquibase.
- ? Add integration tests for moderation status transitions.
- ? Add indexes on visibility_status, moderation_status, created_at, and owner fields.

23. Short Glossary

- ? Clip: A video uploaded by a user.

- ? Post: A public/social share created from a clip.
- ? Library: User's private/personal clip collection.
- ? Feed: Social stream of public approved posts.
- ? Explore: Discovery page for public approved posts.
- ? Visibility status: Whether a clip can be shown publicly.
- ? Moderation status: The review/approval lifecycle of a clip.
- ? Moderation result: Audit row for a scanner/provider result.
- ? Risk score: Numeric moderation risk from scanner or final workflow.
- ? AI reason: Human-readable explanation generated from AI moderation output.